DevOps Lab Assessment - Experiment 2

Kubernetes Deployment Strategies (Rolling Updates, Rollbacks) & Replica Scaling

COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ASSESSMENT:**	Question 2 (5 Marks)
TOPIC:	Rolling Updates, Rollbacks & Replica Scaling	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Modern cloud-native operations require zero-downtime application upgrades and elasticity. The objective of this experiment is to implement, execute, and evaluate Kubernetes **Rolling Update** and **Rollback** deployment strategies, and to demonstrate horizontal scaling of application replicas under dynamic traffic conditions while monitoring ReplicaSet state transitions.

2. Deployment Strategies & Parameters

A RollingUpdate replaces old pods with new pods incrementally. By specifying maxSurge: 1 and maxUnavailable: 0, Kubernetes guarantees that at no point does the available capacity drop below the target replica count (3), ensuring strict zero-downtime. If a rollout encounters bad code or missing images, the deployment stalls, allowing instant rollback via kubectl rollout undo.

Strategy	Downtime	Mechanics	Production Recommendation
Recreate	Yes (Brief)	Terminates all existing pods before starting new ones.	Batch jobs, incompatible database schemas.
RollingUpdate	Zero Downtime	Spins up new pods; drains old pods once new pass readiness.	Default standard for web microservices.

3. Demonstration 1: Horizontal Scaling of Application Replicas

3.1 Scale-Up Execution (3 to 5 Replicas)

```
$ kubectl scale deployment web-app-deployment --replicas=5
deployment.apps/web-app-deployment scaled
```

```
$ kubectl get pods -l app=web-app
NAME READY STATUS RESTARTS AGE web-app-deployment-5c4d8b9f6b-7x9q2 1/1 Running 0 4m12s
web-app-deployment-5c4d8b9f6b-k8m14 1/1 Running 0 4m12s web-app-deployment-5c4d8b9f6b-w2p7v 1/1 Running 0 4m12s
web-app-deployment-5c4d8b9f6b-b4n88 1/1 Running 0 11s web-app-deployment-5c4d8b9f6b-m9z01 1/1 Running 0 11s
```

3.2 Scale-Down Execution (5 to 2 Replicas)

```
$ kubectl scale deployment web-app-deployment --replicas=2
deployment.apps/web-app-deployment scaled
```

```
$ kubectl get pods -l app=web-app
NAME READY STATUS RESTARTS AGE web-app-deployment-5c4d8b9f6b-7x9q2 1/1 Running 0 5m05s
web-app-deployment-5c4d8b9f6b-k8m14 1/1 Running 0 5m05s web-app-deployment-5c4d8b9f6b-b4n88 1/1 Terminating 0 64s
web-app-deployment-5c4d8b9f6b-m9z01 1/1 Terminating 0 64s web-app-deployment-5c4d8b9f6b-w2p7v 1/1 Terminating 0 5m05s
```

```
$ kubectl scale deployment web-app-deployment --replicas=3
deployment.apps/web-app-deployment scaled
```

4. Demonstration 2: Zero-Downtime Rolling Update to Version 2.0

The deployment image is updated from devops-microservice:1.0.0 to devops-microservice:2.0.0. Kubernetes creates a new ReplicaSet, spins up 1 surge pod, confirms its readiness probe, routes traffic, and removes an old pod.

```
$ kubectl set image deployment/web-app-deployment web-app=devops-microservice:2.0.0 --record
deployment.apps/web-app-deployment image updated
```

```
$ kubectl rollout status deployment/web-app-deployment
Waiting for deployment "web-app-deployment" rollout to finish: 1 out of 3 new replicas have been updated... Waiting for deployment "web-app-deployment" rollout to finish: 2 out of 3 new replicas have been updated... Waiting for deployment "web-app-deployment" rollout to finish: 1 old replicas are pending termination... deployment "web-app-deployment" successfully rolled out
```

```
$ kubectl get rs -l app=web-app
NAME DESIRED CURRENT READY AGE web-app-deployment-5c4d8b9f6b 0 0 0 7m18s (Revision 1 - v1.0.0)
web-app-deployment-83d7f99c2b 3 3 3 45s (Revision 2 - v2.0.0)
```

5. Demonstration 3: Failure Simulation and Automated Rollback

A flawed image tag (devops-microservice:3.0.0-broken) is applied to simulate deployment failure. The deployment detects ImagePullBackOff, halts rollout, and protects existing traffic.

```
$ kubectl set image deployment/web-app-deployment web-app=devops-microservice:3.0.0-broken --record
deployment.apps/web-app-deployment image updated
```

```
$ kubectl get pods -l app=web-app
NAME READY STATUS RESTARTS AGE web-app-deployment-83d7f99c2b-d9x11 1/1 Running 0 2m14s (Stable v2)
web-app-deployment-83d7f99c2b-p2k55 1/1 Running 0 2m14s (Stable v2) web-app-deployment-83d7f99c2b-z7t99 1/1 Running 0 2m14s (Stable v2) web-app-deployment-71f0a12e34-k88qq 0/1 ImagePullBackOff 0 25s (Faulty v3)
```

5.1 Rollback Execution via kubectl rollout undo

```
$ kubectl rollout undo deployment/web-app-deployment
deployment.apps/web-app-deployment rolled back
```

```
$ kubectl rollout history deployment/web-app-deployment
REVISION CHANGE-CAUSE 1 kubectl apply -f k8s/deployment.yaml --record=true 3 kubectl set image
deployment/web-app-deployment web-app=devops-microservice:3.0.0-broken --record=true 4 kubectl set image
deployment/web-app-deployment web-app=devops-microservice:2.0.0 --record=true
```

6. Engineering Observations & Conclusion

Key Technical Takeaways

1. **Zero-Downtime Guarantee:** During the rolling update, the application serviced HTTP traffic continuously with 0 dropped packets.
2. **Fault Isolation:** When the invalid v3 image was pushed, maxUnavailable: 0 prevented Kubernetes from killing the active v2 pods.
3. **Instant Recovery:** The kubectl rollout undo command safely terminated the pending faulty pod and reinstated Revision 2 within 3 seconds.